Advanced Concept

* Functional Programming :-

* lambda :-

→ It is anonemouse function that is used to perform small operation.

→ lambda is a keyword.

Syntax :-

i) Var_name = lambda args : Expression

ii) Var_name = lambda args : TSB if cond else FSB : Expression

Ex WAP to check whether no. is even or odd using lambda function.

```
even = lambda n : n%2==0
>>even(7)    O/P False
```

Ex WAP to print sum of two no. by using lambda.

```
s = lambda a,b : a+b
print(s(2,3))    O/P 5
```

Ex WAP to fetch the even no. from the list.

```
l = eval(input('enter the number:'))
even = lambda(map n :
```

Ex even_odd = lambda n : 'even' if n%2==0 else 'odd'

Ex WAP to check whether the first char of string is vowel or not.

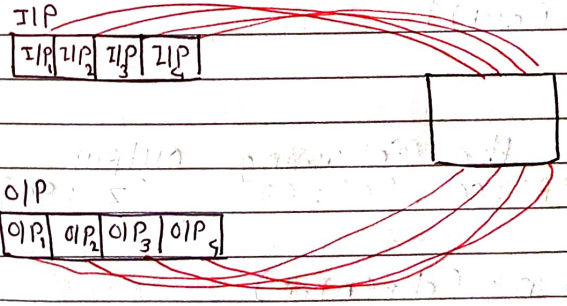
```
chk_vowel = lambda s : 'vowel' if s[0] in 'AEIOUaeiou' else 'Not vowel'
```

* map() :-

→ It is a inbuild function that is used to perform operation on each & every value that is present inside the collection.

Syntax :- Var_name = map(function_name,collection)
print(list(Var_name))

•Note :- Var_name does not have proper structure to display output for that reason we need type casting.



Note :- Length of I/P collection is same as length of O/P collection.

Ex Create a list to store the square values from the collection of integer 1 to 10

```
Sqr = lambda n : n**2
IP = [1,2,3,4,5,6,7,8,9,10]
out = ( Sqr, IP)
print(list(out))
```

OIP [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

OR

```
for i in out :
    print(i)
```

Ex WAP to store the factorial from the list of list having positive integer.

```
def fact(n):
    f=1
    for i in range (1,n+1):
        f = f*i
    return f.
IP = [1,2,3,4,5,6,7,8,9,10]
out= map( fact , IP)
print ( list(out))
```

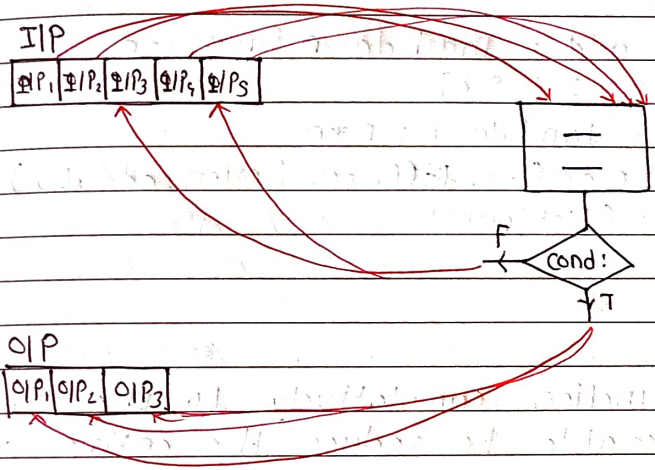
Ex WAP to get the following output
{'A': 65, 'B': 66, 'C': 67 ..... 'Z': 90}

```
d= lambda n : (chr(n),n)
out= map(d,range(65,91))
print( dict(out))
```

* Filter() :-

→ It is a inbuild function that is used to fetch required data from collection.

Syntax:- Var name = filter(fname, collection)
print(list (var name))



Note:- The length output collection is same or lessthan input collection.

Ex WAP to fetch the even no. from the list.

```
even = lambda n : n%2 == 0
L = [1, 2, 3, 4, 5, 6, 7, 8]
out = filter (even, L)
print ( list(out))
```

Ex WAP to fetch the even no. from the list & store its square.

```
even_odd = lambda n : n%2 == 0
z = filter(even_odd, t)
l = eval(input('Enter the list:'))
out = filter(even_odd, l)
sqr = lambda m : m*m
ans = map(sqr, out)
print(list(ans))
```

OR

```
even_odd = lambda n : n%2 == 0
l = [1,2,3,4,5,6]
sqr = lambda n : n*n
out = map(sqr, list(filter(even_odd, l)))
print(list(out))
```

* **reduce :-**

→ It is function cumulatively to the elements of an iterable to reduce the entire sequence down to a single final value.

Syntax: from functool import reduce
reduce(function, iterable [, initializer])

Ex
```
from funtool import reduce
add = lambda a,b : a+b
l = [1,2,5,6]
s = reduce(add, l)
print(s)
```
O/P 14

* **Comprehension :-**

→ Reducing the number of line of instruction to create collection that is known as comprehension.

→ There are three types
  * List Comprehension
  * Set Comprehension
  * Dictionary Comprehension

i) **List Comprehension :-**

→ Creating a new list by reducing the number of lines of instructions that is known as list comprehension.

Syntax :-
```
var_name = [var for var in collection if cond]
```
OR
```
[TSB if cond else FSB for var in collection]
```
OR
```
[var1, var2 for var1 in collection1
 for var2 in Collection2]
```

Ex To create a list that consist 1 to 10 integer values
```
out = [i for i in range(1,11)]
print(out)
```
O/P [1,2,3,4,5,6,7,8,9,10]

Ex Create a list that contains only integer value from the given list.

```
l = [3.4, 'hii', 0+4j, 'Rohit']
out = [i for i in l if type(i) == int]
print(out)        o/p [3,4]
```

Ex WAP to get the following output
op [('A',1), ('A',2), ('A',3), ('B',1), ('B',2), ('B',3)]

```
out = ([i,j for i in 'AB' for j in range(1,4)])
print(out)
```

Ex I/P [1,2,3,4,5,6]
o/p [False, True, False, True, False, True]

```
out = [True if i%2==0 else False for i in
                                range(1,7)]
print(out)
```

* Set Comprehension :-
→ Creating the new set by reducing the number of lines of instruction that is known as Set comprehension

• Syntax :

```
Var_name = {var for var in collection
                        if condition}
```

OR

```
{ TSB if cond else FSB for var
                    in collection}
```

OR

```
{ var1, var2 for var1 in collection1
            for var2 in collection2}
```

Ex WAP to get following output
IP = [6, 7, 'hii', 'hello', 'Pranav']
OP = {'hii', 'hello', 'Pranav'}

```
print({i for i in ~~range~~ IP if type(i)==str})
```

* zip() :-
→ It is a inbuild function that is used to perform iteration on multiple collection inside the single for loop.

Syntax :

```
for var1, var2 ... varn in zip(coll1, coll2
                            .. colln):
```

SB

Note :- No. of variable = No. of collection
Note :- No of iteration is directly proportional to minimum length of the collection

Ex
```
for i,j in zip('AB', [6,7,2]):
        print(i,j)
```
o/p A 6
B 7

Ex
```
for i in zip('AB', [6,7,2]):
        print(i)
```
o/p ('A', 6)
('B', 7)

* Dictionary Comprehension:-

→ Creating a dictionary by reducing the number of lines of instruction that is known as dictionary comprehension.

Syntax:

Var_name = {key:value for var in collection if condition}

{k:v for var1, var2 in zip(coll1, coll2)}

{k1:v1 if condition else k2:v2 for var1, ~~var2~~ in collection}

Ex WAP to get following output.
{1:1, 2:4, 3:9, 4:16}

```
print({i:i**2 for i in range(1,5)})
```

Ex WAP to get following output
IP hai hello haii

```
s= input('Enter the string:')
print({i:len(i) for i in s if len(i)==0})
```

o/p {'haii':4}

* Decorator:-

→ It is function that is used to give some extra information to the main task without modifing it.

→ There are two types
* User defined decorator
* Inbuild decorator.

* User defined decorator:-

→ The decorator that is defined by the users as per there requirement that is known as user defined decorator.

Syntax:- Creating decorator. → address of main task/ function.

```
def deco_name(func):
    def inner(*args, **kwargs):
        Pre task
        func(*args, **kwargs)
        Post task
    return inner
```

Syntax:- Used the decorator

```
@deco_name  → Main task
def func(args):
    SB
func(values)
```

Ex

```
def insta(func):
    def inner(*args, **kwargs):
        print('login:')
        func(*args, *kwargs)
        print('logout')
```

```
    return inner

@insta
def A_user():
    print('Story Post')

@insta
def B_user():
    print('like the story')

@insta
def C_user():
    print('Comment on the story')
```

A_user() o/p Login
              Story Post
              Logout

Ex Create a decorator to delay any program to run for 5 sec.

```
def delay(func):
    def inner(*args, **kwargs):
        impor time
        time.sleep(5)
        func(*args, *kwargs)
    return inner

@delay
def rev_list(l):
    print(l[::-1])
```

```
>> rev_list([8,5,6])
```

Ex Create a decorator to find the total time execution to run any program.

```
def find_time(func):
    def inner(*args, **kwargs):
        import time
        st = time.time()
        func(*args, *kwargs)
        end = time.time()
        print(end-st)
    return inner

@find_time
def rev_list(l):
    print(l[::-1])
```

* Generator :-

→ It is a function that is used to create a collection by using yield keyword.

Ex
```
def sam:
    print('hii')
    yield 1
    print('hello')
```

sam ⟶ function address
sam() ⟶ generator address
list(sam()) ⟶ typecasting to fetch value

• we cannot typecast in the form of string otherwise return address of generator.

```
for i in sam():
    print(i)
```

O/P hii
1
hello

Note:- If the function holding return as well as yield keyword the priority is given to the yield keyword.

Ex.
```
def sam:
    print('hiiiii')
    return 1
    print('hellllloo')
    yield 2
```

list(sam()) O/P hiiiii
[]

* Difference between Yield & return keyword

	return	Yield
i) Function type	Regular function	i) Generator function
ii) Execution	ii) Exits immediatly	ii) suspends & saves local states.
iii) Frequency	iii) Execute exactly once	iii) can execute multiple times.
iv) OUTPUT	iv) A specif data type	iv) A generator object
v)	v) Memory usage is high (store entire dataset in RAM)	v) Memory usage is low (computes items on demand)

Ex. WAP to create a list containing the sqr of integer 1 to 10 by using generator

```
def sqr():
    for i in range(1,11):
        yield i**2
```

~~sqr~~ list(sqr()) O/P [1,4,9,16,25,36,49,64,81,100]

Ex. WAP to extract the all string value from the given list by using generator.

```
def str_value(l):
    for i in l:
        if type(i) == str:        # if isinstance(object, class):
            yield i               #     if isinstance(i, str):

list(str_value(['hii',1,8.0,0+4j,'hello']))
```

O/P ['hii', 'hello']

Ex. WAP to get following output.
O/P {'A':65, 'B':66, ... 'Z':90}

```
def get_dict():
```

```
for i in range(65, 91):
    yield (chr(i), i)

print(dict(get_dict()))
```

Ex WAP to get following output
IP = ['hii', 10, 'hello', [15, 'bye', 'hii']]
OP = ['hii', 'hello', 'bye', 'hii']

```
def get_output(l):
    for i in l:
        if type(i) == str:
            yield i
        elif type(i) in [list, set, tuple]:
            for j in i:
                if type(j) == str:
                    yield j

list(get_output())
```

Ex WAP to get first 10 fibonacci number by using generator.

```
def fibo(s, e):
    a=0, b=1
    for i in range(s, e+1):
        a, b = b, a+b
        yeild a
        a, b = b, a+b
```

```
list(fibo(1, 10))
```

Ex WAP to get palindrome number in given number range by using generator

```
def get_palindrome(a, b):
    for i in range(a, b+1):
        if i == int(str(i)[::-1]):
            yield i

list(get_palindrome(1, 100))
```

OR

```
def get_palindrome(s, e):
    for i in range(s, e+1):
        temp = i
        rev = 0
        while i > 0:
            ld = temp % 10
            rev = rev * 0 + ld
            temp = temp // 10
        if i == rev:
            yield i

print(list(get_palindrome(1, 100)))
```

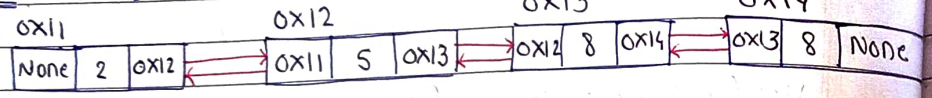
* Iterator :-

→ The function that is used to perform iteration on the collection.

→ There are two inbuild function: that is used in iterator.

* iter()
* next()

Ex l = [2, 5, 8, 7]



• iter() :-

→ It is inbuild function that is used to fetch the address of first value from the collection

• next() :-

→ It is inbuild function that is used to fetch the current address value and point to the next address.

Ex l = [2,5,8,7]
i = iter(l)
next(i) o/p 2
next(i) o/p 5
next(i) o/p 8
next(i) o/p 7
next(i) exception : StopIteration.

* File Handling :-

→ Writing the data into the file or reading the data from the file.

Syntax: Var_name = open('file name / path', 'mode')

• Mode :-

i) w (write) :
w+ (write & read)
wb (write binary)
wb+ (write, read binary)

ii) r (read) :
r+ (read, write)
rb (read binary)
rb+ (read, write binary)

iii) a (append) :
a+ (append, read)
ab (append binary)
ab+ (append, read binary)

• Write (w) :-

→ It is used to write data into the file.

i) write() :

→ It is inbuild function that is used to write single line data into the file.

ii) writeline() :-

→ It is inbuild function that is used to write

then it will create.

write multiple line of data into the file

Syntax: Var_name.write(data)
Var_name.close()

Ex
```
f = open('Demo.txt', 'w')
f.write('Good Afternoon')
f.close()
```

Note :- To avoid the unicode error we have to used 'r' string.

Ex
```
f = open(r'' C:\Users\..... '', 'w')
f.write('Hello')
f.close()
```

1]
```
with open('Demo.txt', 'w') as f:
    f.write('happy independence Day!')

with open('Demo2.txt', 'r') as f:
    #print(f.read())
    # read entire data from this file.

with open('Demo2.txt', 'r') as f:
    # print(f.read())

# read entire data from the file.
```

* read mode(r) :-
→ It is inbuild function that is used to read entire line data into the file (Demo2.txt)

1) readline() :- Read single line from file

2) readlines() :- Read entire data from the file & store in list.

Ex
```
with open('file_name', 'w') as f:
    f.write('Hi')

with open('file_name', 'r') as f:
    print(f.read())
    print(f.readlines())
    print(f.readlines())
```
O/P ['Hello', 'Bye', 'Hello', 'Hii']

* Seek() :-
→ It is inbuild function that is used to point the cursor to the particular index position

Ex
```
with open('Demo.txt', 'r') as f:
    f.seek(6)
    print(f.read())

with open('viratKohli.jpg', 'rb') as VK:
    a = VK.redd()
```

remove empty spa
between two lines.

```
with open('New_virat_kohli.png', 'wb') as nvk:
    nvk.write(a)

with open('redfort.png', 'rb') as rf:
    res = rf.read()

with open('flag.png', 'wb') as wf:
    wf.write(res)
```

* File Handling with CSV file :-

- Syntax :- import csv (write mode).

```
with open('file.csv/path', 'w') as var_name1:
    var_name2 = csv.writer(var_name)
    var_name2.writerow([])
        OR
    var_name2.writerows([[],[]])
```

Ex

```
import csv
with open('first.csv', 'w', newline='') as
                                    file:
    fw = csv.writer(file)
    fw.writerow(['Name', 'Roll', 'Age'])
    fw.writerows([['xyz', 2, 21], ['par', 3, 22]])
```

- Syntax :- Read Mode.

```
import csv
with open('file.csv'/Path , 'r') as var_name:
    var_name2 = csv.reader(var_name)
    print(list(varname2))
```

Ex

```
import csv
with open('first.csv', 'r') as f :
    fr = csv.reader(f)
    print(list(fr))
        OR
    for i in fr :
        print(i)
```

* Parsing Technique :-

→ It is a way to encrypt or decrypt the data.
→ There are two types of parsing technique.
  * JSON Parsing technique
  * Pickle Parsing technique

° JSON Parsing Technique :-

→ It is a way to store the data in the form of string data

° Syntax of Encryption :-

```
import JSON
Encrypt_data = JSON.dumps(data)
```

° Syntax of Decryption :-

```
import JSON
decrypt_data = JSON.loads(Encrypt_data)
```

- dumps() :- It is inbuild function that is used to encrypt the data.

- loads() :- It is inbuild function that is used to decrypt the data.

Ex Encryption
```
import json
data = [10,20, 'hii', 'hello']
encrypt_data = json.dumps(data)
print(type(data))          o/p list
print(type(encrypt_data))  o/p str
```

Ex Decryption
```
decrypt_data = json.load(encrypt_data)
print(type(encrypt_data))  o/p str
print(type(decrypt_data))  o/p list
```

- Pickle Parsing Technique :-

→ Pickling is the process of converting a Python object into the byte stream so that it can be saved to a file or transmitted over a network.

- Syntax for Encryption
```
import pickle
encrypt_data = pickle.dumps(data)
```

- Syntax for Decryption
```
import pickle
decrypt_data = pickle.loads(encrypt_data)
```

Ex
```
import pickle
data = 'Hello World'
en_data = pickle.dumps(data)
print(en_data)
# decryption
dc_data = pickle.loads(en_data)
print(dcdata)
```

* Exception Handling :-

→ Handling the runtime error that is known as Exception Handling.

→ To make the code more smooth we are using exception handling.

→ There are three way for exception handling
- * Specific Exception Handling
- * Generic Exception Handling
- * Default Exception Handling.

- Specific Exception Handling :-

→ When users know which type of exception will occur during the runtime than we are ~~sp~~ using specific exception handling.

Syntax :-
```
try :
    Program
except exception_name:
    solution
```

Ex
```
try :
    a = int(input('Enter the N1:'))
    b = int(input('Enter the N2:'))
    print(a/b)
except ZeroDivisionError:
    print("Number can't div by zero")
print('Hii')
```

O/P N1: 10 N2: 0
Number can't div by zero
Hii

• Drawback :- It will not handled all type of exceptions.

* Generic Exception Handling :-
→ Will Handled all types of exception.

Syntax:-
```
try :
    program
except Exception as var:
    Solution
```
(Exception → inbuild class)

Ex
```
try :
    a = int(input('Enter N1:'))
    b = int(input('Enter N2:'))
    print(a/b)

except Exception as e:
    print(e)
```

• Drawback :- It will not handled keyboardInterrupt error.

* Default Exception :-
→ Default Exception will handle all the type of exception including keyboard Interrupt exception.

Syntax:
```
try :
    Program
except :
    Solution
```

Ex
```
try :
    while True:
        print('Hello')

except :
    print('Exception Handled')
```

→ There are Mainly 4 keywords that is used in exception handling.

i) try
ii) except
iii) else
iv) finally.

• order of execution :-
• try → except → finally
• try → else → finally.

Ex
```
try :
    s=0
    for i in range(0,6):
        s=s/i
except :
    s=5
else :
    s=7
finally:
    print(s)
```
O/P 5

* Custom Exception :-

→ Creating the user defined exception that is known as Custom Exception

→ By using raise keyword we are creating User defined exception.

Ex
```
class UserError(Exception):
    pass

n= int(input('Enter the No. of product'))
if n<=0:
    raise UserError('Product out of stock')
else :
    print('Ordered successfully').
```

* Assertion :-

→ It is a process of creating used defined exception without using error name.

Syntax :- assert condition , 'msg'

Ex
```
s1 = input('Enter str1')
s2 = input('Enter str2')
assert len(s1)==len(s2) , 'length is different'
print(s1+s2)
```

O/P ① hii
hey
hiihey

② hii
hello
length is different.

* Regular Expression :-

→ Write the expression to match pattern to get the required data from the huge data.

Syntax :-

[A-Z]
[a-z]
[0-9] or \d
[A-Z a-z]
[A-Z a-z 0-9] or \w
[A-Z] {n}
[A-Z] {m,n}

[\w+] → words
[\w*] → words, space.

\ → zero or more occurences

+ → one or more occurences

* → zero or more occurences

? → zero or one occurences.

^ → start of string

$ → end of string

. → Any single character except newline.

import re :-

i) findall() :-

→ It will return all match records from given data.

ii) finditer() :-

→ It will find the index of match data

Syntax:- ~~import re~~

i) findall() :

```
var_name = re.findall('Pattern', 'data')
```

ii) finditer() :

```
var_name = re.finditer('Pattern', 'data')
```

Ex Expression to match phone number.

→ [6-9] [0-9] {9} (10 digits; 6-9, 0-9, 9)

→ [6-9] \d {9}

```
import re
s = input('Enter phone no.:')
result = re.findall('[6-9][0-9]{9}', s)
print(result)
```

Ex Write an Expression to match data

DD/MM/YYYY	[0-9]{2}/[0-9]{2}/[0-9]{4}
OR	
DD/MON/YYYY	\d{2}/\d{2}/\d{4}

```
import re
s = input()
res = re.findall('\d{2}/\d{2}/\d{4}')
print(res)
out = re.finditer('\d{2}/\d{2}/\d{4}')
print(list(out))
```

Ex Patter to match time.

4:15 06:30
07:45 4:03

\d{1,2}\:\d{2}

```
import re
s = input()
res = re.findall('\d{1,2}\:\d{2}', s)
out = re.finditer('\d{1,2}\:\d{2}', s)
print(res)
print(list(out))
```

Ex To match website

www.\[a-z]+\.[a-z]{2,3}

Ex To match USN
2AB16EC013
\d [A-Z] {2} \d {2} [A-Z] {2} \d {3}

Ex Expression to match the pan number
ABCDF1239T
[A-Z] {5} \d {4} [A-Z]

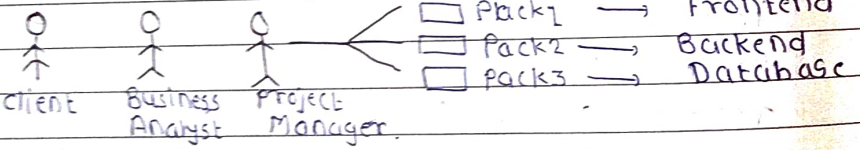
Ex To match email ID
abcd@gmail.com
\w+\@\w+\. [a-z A-Z] {2,3}

OR

abcd.efg@gmail.com
\w+\.\w+\@\w+\. [A-Z a-z] {2,3}

Common Expression
\w+.?\w*\@\w+\. [A-Z a-z] {2,3}

* Package Architecture :-



→ It is the phenomenon of dividing compute project into packages.

• Package :-
→ It is the folder which consists of python files ('n' number of files)

Package	Module
i) Folder	i) File
ii) pip install package name or	ii) import module is compulsory

• import :- are used to bring code from modules into program.

• from :- import specific function, classes or variables from a modules.

Ex import math
math.sqrt(64) O/P 8.0
math.factorial(5) O/P 120

from math import *
sqrt(8) 2.82
pi 3.145

• Module :-
→ Python file containing functions, classes and variables.

Syntax: import module_name
module_name.var / function_name()

from module_name import *
var / fname()

from module_name import var, fname()
var / fname()

• randome module :-

→ The random module is used to generate the random number, select random item, shuffle data.

i) random() :- Generate a random float between 0.0 & 1.0

ii) randint(a,b) :- Generate a random integer between a & b

iii) choice() :- Select a random element from a sequence

iv) choices() :- Select a multiple random elements.

v) shuffle() :- Randomly rearrange a list.

• Calender Module :-

→ The calender module is used to work with dates, months, years & calenders

i) Display a Month calender:
- Calender.month(year, month)

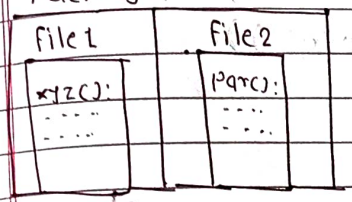
ii) Display full year :
calender.calender(year)

iii) Check leap year:
calender.isleap(year)

• User defined Package :-

→ It is a package that is created by the user as per their requirement.

Package.



import file1
file1.mname() / var.

Ex file1

```
def valid_pw(s):
    if len(s)>=8:
        u=0, l=0, sc=0, d=0
        for i in s:
            if 'A'<=i<='Z':
                u+=1
            elif 'a'<=i<='z':
                l+=1
            elif '0'<=i<='9':
                d+=1
            else:
                sc+=1
        if u>=1 and l>=1 and d>=1 and sc>=1:
            return True
        return False

def valid_un(s):
    if '.' in s:
        return True
```

```
else :
    return False

if __name__ == "__main__" :
    print(valid_pw("Abc@1234"))
    print(valid_un("abc.efg"))
```

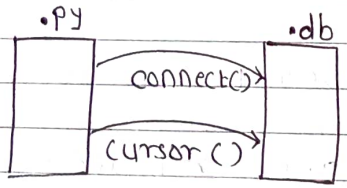
file2

```
import file1
un = input('Enter username :')
pw = input('Enter password:')
print( file1.valid_un(un))
print( file1.valid_pw(pw))
```

* SQL Connection :-

- Sqlite3 :-

→ The good thing about Sqlite3 is that it is already included in python. You do not need to install a separate package.



- connect() :- It used to establish a connection between python program and the SQlite database.

- cursor() :- After connecting to the database cursor() creates an object that allows to execute SQL queries.

Syntax :-

```
import sqlite3
var1 = sqlite3.connect('data.db')
var2 = var1.cursor()
var1.execute('query')
var1.commit()
var1.close()
```

- execute() :- It is used to execute the SQL query.

- Connect() :- Connect python to database.

- Syntax for creating Table :-

```
cursor.execute(''' CREATE TABLE TEMP
                   TABLE_NAME ( col_name1
                   'datatype constraint.....,
                   col_name2 datatype constrai-
                   nt);''')
```

- Syntax for insert the data :-

```
cursor.execute(''' INSERT INTO TABLE_NAME
                   values (v1, v2, v3 ... vn);''')
```

- Syntax for extract the data :-

```
cursor.execute(" SELECT * FROM TABLE_NAME ")
```

Ex

```
import sqlite3

d = sqlite3.connect('data.db')
c = d.cursor()
d.execute("""CREATE TABLE EMP ( id number(10),
            ename varchar(20), esal number(5))
            """)

c.execute(" INSERT INTO EMP VALUES (
            1, 'xyz', 5000)")

c.execute(" INSERT INTO EMP VALUES
            (2, 'qpr', 6000)").

d.commit()
d.close()
```

Ex Fetch the data

```
import sqlite3
d = sqlite3.connect('data.db')
c = d.cursor()
result = c.execute("SELECT * FROM EMP")
print( list(result))
print( list result.fetchall()) → get all rows
print(result.fetchone()) → get one row

d.close()
```